

OS LAB 12

24K-0559

BSCS 4H

CODE WORKOUT #1).

OUTPUT:

```
new-user@new-user-VirtualBox:~$ nano q1.c
new-user@new-user-VirtualBox:~$ gcc q1.c -o p
new-user@new-user-VirtualBox:~$ ./p
Program running... Press Ctrl+C
^C
SIGINT caught! (Signal number: 2)
```

```
GNU nano 7.2 q1.c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <stdlib.h>

void sigint_handler(int signum)
{
    printf("\nSIGINT caught! (Signal number: %d)\n", signum);
}

int main() {
    if (signal(SIGINT, SIG_IGN) == SIG_ERR) {
        printf("Error setting up signal handler\n");
        return 1;
    }

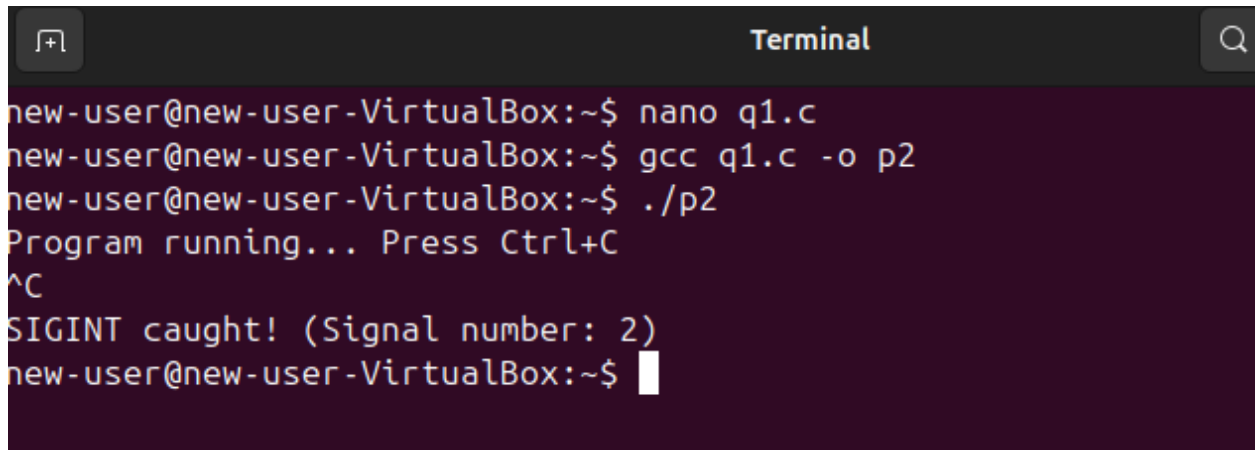
    printf("Program running... Press Ctrl+C\n");

    while (1) {
        sleep(1);
    }

    return 0;
}
```

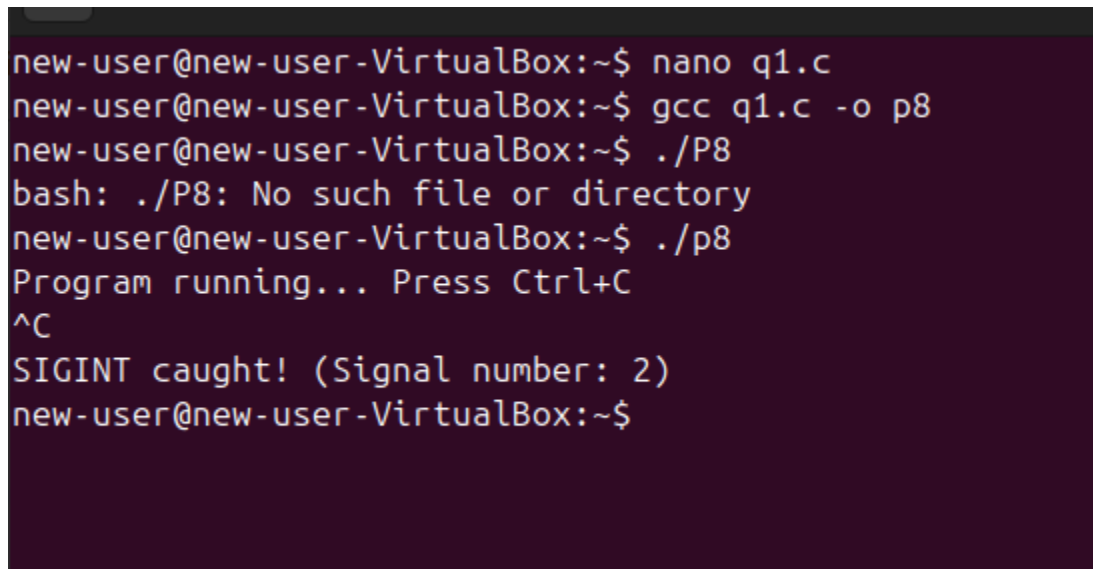
THIS IS USED TO IGNORE CTRL+C COMMAND.

```
Terminal
new-user@new-user-VirtualBox:~$ nano q1.c
new-user@new-user-VirtualBox:~$ gcc q1.c -o p7
new-user@new-user-VirtualBox:~$ ./p7
Program running... Press Ctrl+C
^C^C
```

A terminal window titled "Terminal" with a dark background. It shows a user at a prompt "new-user@new-user-VirtualBox:~\$". The user enters "nano q1.c", then "gcc q1.c -o p2", and finally "./p2". The program runs and displays "Program running... Press Ctrl+C". The user presses Ctrl+C, indicated by "^C", and the terminal shows "SIGINT caught! (Signal number: 2)". The prompt returns to "new-user@new-user-VirtualBox:~\$".

```
new-user@new-user-VirtualBox:~$ nano q1.c
new-user@new-user-VirtualBox:~$ gcc q1.c -o p2
new-user@new-user-VirtualBox:~$ ./p2
Program running... Press Ctrl+C
^C
SIGINT caught! (Signal number: 2)
new-user@new-user-VirtualBox:~$
```

NOW STOPPED AFTER ADDING EXIT COMMAND.

A terminal window showing a user at a prompt "new-user@new-user-VirtualBox:~\$". The user enters "nano q1.c", then "gcc q1.c -o p8", and finally "./P8". The terminal shows an error: "bash: ./P8: No such file or directory". The user then enters "./p8", and the program runs, displaying "Program running... Press Ctrl+C". The user presses Ctrl+C, indicated by "^C", and the terminal shows "SIGINT caught! (Signal number: 2)". The prompt returns to "new-user@new-user-VirtualBox:~\$".

```
new-user@new-user-VirtualBox:~$ nano q1.c
new-user@new-user-VirtualBox:~$ gcc q1.c -o p8
new-user@new-user-VirtualBox:~$ ./P8
bash: ./P8: No such file or directory
new-user@new-user-VirtualBox:~$ ./p8
Program running... Press Ctrl+C
^C
SIGINT caught! (Signal number: 2)
new-user@new-user-VirtualBox:~$
```

PLACING SIGKILL TO KILL IT.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <stdlib.h>

void sigint_handler(int signum)
{
    printf("\nSIGINT caught! (Signal number: %d)\n", signum);
    exit(0);
    raise(SIGKILL);
}

int main() {

    if (signal(SIGINT, sigint_handler) == SIG_ERR) {
        printf("Error setting up signal handler\n");
        return 1;
    }

    printf("Program running... Press Ctrl+C\n");
```

```
SIGINT caught! (Signal number: 2)
new-user@new-user-VirtualBox:~$ ./p8
Program running... Press Ctrl+C
^C
SIGINT caught! (Signal number: 2)
new-user@new-user-VirtualBox:~$
```

```
Terminal × Terminal ×
new-user@new-user-VirtualBox:~$ ps -ef | grep prog
new-user      5179      5170  0 13:06 pts/1    00:00:00 grep prog
new-user@new-user-VirtualBox:~$ ps -ef | grep prog
new-user      5190      5170  0 13:07 pts/1    00:00:00 grep prog
new-user@new-user-VirtualBox:~$
```

PROGRAM KILLED.

```
new-user@new-user-VirtualBox:~$ gcc q1.c -o p9
new-user@new-user-VirtualBox:~$ ./p9
Program running... Press Ctrl+C
^C
new-user@new-user-VirtualBox:~$
```

```
GNU nano 7.2 q1.c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <stdlib.h>

void sigint_handler(int signum)
{
    printf("\nSIGINT caught! (Signal number: %d)\n", signum);
    exit(0);
    raise(SIGKILL);
}

int main() {
    if (signal(SIGUSR1, sigint_handler) == SIG_ERR) {
        printf("Error setting up signal handler\n");
        return 1;
    }

    printf("Program running... Press Ctrl+C\n");

    while (1) {
        sleep(1);
    }

    return 0;
}
```

THE SIGUSR1 IS BEEN USED TO KILL THE PROGRAM.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <stdlib.h>
```

```
void sigint_handler(int signum)
{
    printf("\nSIGINT caught! (Signal number: %d)\n", signum);
    exit(0);
    raise(SIGKILL);
}
```

```
int main() {
```

```

if (signal(SIGUSR1, sigint_handler) == SIG_ERR) {
    printf("Error setting up signal handler\n");
    return 1;
}

printf("Program running... Press Ctrl+C\n");

while (1) {
    sleep(1);
}

return 0;
}

```

PROBLEM Q1A).

```

new-user@new-user-VirtualBox:~$ nano q1A.c
new-user@new-user-VirtualBox:~$ nano q1A.c
new-user@new-user-VirtualBox:~$ gcc q1A.c -o p6
new-user@new-user-VirtualBox:~$ ./p6
^C Ctrl+C (SIGINT) received. Exiting...

```

This program demonstrates handling of the SIGINT signal using the sigaction() system call, which is a modern alternative to signal(). A structure sigaction is used to define:

The signal handler function And When the user presses Ctrl+C:

the SIGINT signal is generated.

```
new-user@new-user-VirtualBox:~$ nano q1A.c
new-user@new-user-VirtualBox:~$ gcc q1A.c -o p5
new-user@new-user-VirtualBox:~$ ./p5
^C
new-user@new-user-VirtualBox:~$
```

USING SIGTERM TO TERMINATE THE PROGRAM.

```
new-user@new-user-VirtualBox:~$ nano q1A.c
new-user@new-user-VirtualBox:~$ gcc q1A.c -o p2
new-user@new-user-VirtualBox:~$ ./p2
^C
```

USING SIGN IN

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
```

```
void sigint_handler(int signum) {
    printf("Ctrl+C (SIGINT) received. Exiting...\n");
    exit(signum);
}
```

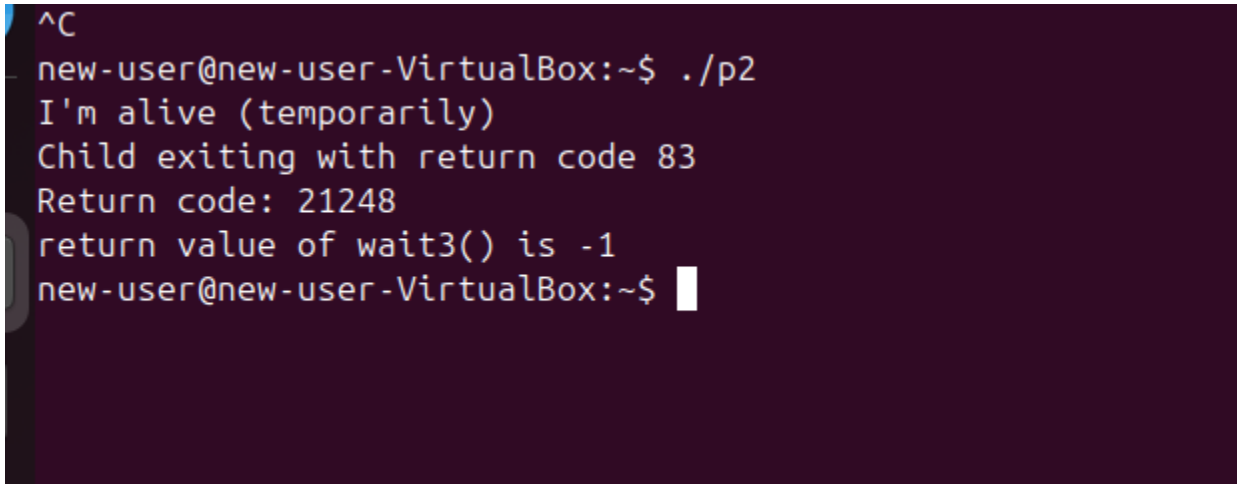
```
int main() {
    struct sigaction sa;
    sa.sa_handler = SIG_IGN;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    if (sigaction(SIGTERM, &sa, NULL) == -1) {
        perror("sigaction");
        return EXIT_FAILURE;
    }
```

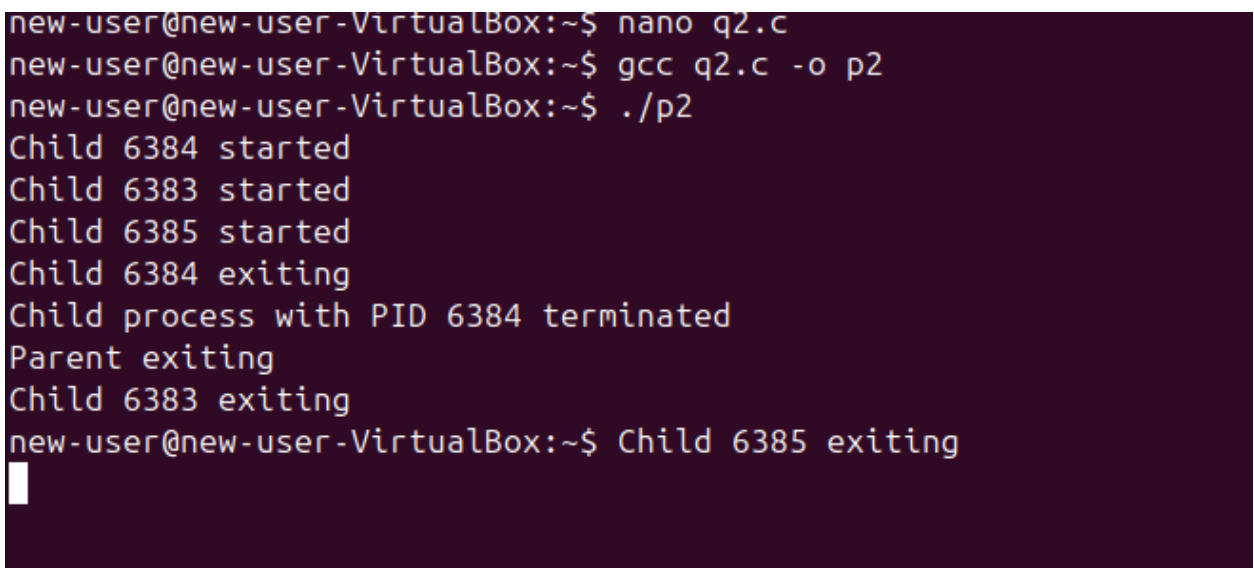


```
while (1) {  
}  
  
return EXIT_SUCCESS;  
}
```

PROBLEM 2).



```
^C  
new-user@new-user-VirtualBox:~$ ./p2  
I'm alive (temporarily)  
Child exiting with return code 83  
Return code: 21248  
return value of wait3() is -1  
new-user@new-user-VirtualBox:~$
```



```
new-user@new-user-VirtualBox:~$ nano q2.c  
new-user@new-user-VirtualBox:~$ gcc q2.c -o p2  
new-user@new-user-VirtualBox:~$ ./p2  
Child 6384 started  
Child 6383 started  
Child 6385 started  
Child 6384 exiting  
Child process with PID 6384 terminated  
Parent exiting  
Child 6383 exiting  
new-user@new-user-VirtualBox:~$ Child 6385 exiting
```

This program demonstrates handling of the SIGCHLD signal, which is generated when a child process terminates. The parent process creates multiple child processes using fork().

When a child exits:

- The OS sends a SIGCHLD signal to the parent
- Inside the handler:
- `waitpid(-1, &status, WNOHANG)` is used
- It collects terminated child processes without blocking
- This prevents the creation of zombie processes.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
```

```
void sigchld_handler(int signum) {
    pid_t pid;
    int status;

    while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
        printf("Child process with PID %d terminated\n", pid);
    }
}
```

```
int main() {

    signal(SIGCHLD, sigchld_handler);

    for (int i = 0; i < 3; i++) {
        pid_t pid = fork();

        if (pid == 0) {

            printf("Child %d started\n", getpid());
```

```

        sleep(2);
        printf("Child %d exiting\n", getpid());
        exit(0);
    }
}

sleep(5);

printf("Parent exiting\n");

return 0;
}

```

PROBLEM 3).

```

new-user@new-user-VirtualBox:~$ nano q3.c
new-user@new-user-VirtualBox:~$ gcc q3.c -o p2
new-user@new-user-VirtualBox:~$ ./p2
Thread 1 started (TID: 139243271747264)
Thread 3 started (TID: 139243254961856)
Thread 2 started (TID: 139243263354560)

Sending signal to PROCESS
Signal 10 handled by thread 139243275503424

Sending signal to THREAD 3
Signal 10 handled by thread 139243254961856
new-user@new-user-VirtualBox:~$ █

```

```

#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <signal.h>

#include <unistd.h>

pthread_t threads[3];

```

```
void handler(int sig) {  
    printf("Signal %d handled by thread %lu\n", sig, pthread_self());  
}
```

```
void* worker(void* arg) {  
    int id = *(int*)arg;  
  
    printf("Thread %d started (TID: %lu)\n", id, pthread_self());  
  
    while (1) {  
        sleep(1);  
    }  
  
    return NULL;  
}
```

```
int main() {  
    int ids[3] = {1, 2, 3};  
  
    signal(SIGUSR1, handler);  
  
    for (int i = 0; i < 3; i++) {  
        pthread_create(&threads[i], NULL, worker, &ids[i]);  
    }
```

```
sleep(2);

printf("\nSending signal to PROCESS\n");
kill(getpid(), SIGUSR1);

sleep(2);

printf("\nSending signal to THREAD 3\n");
pthread_kill(threads[2], SIGUSR1);

sleep(2);

return 0;
}
```

```
new-user@new-user-VirtualBox:~$ gcc q3.c -o p4
new-user@new-user-VirtualBox:~$ ./p4
Thread 1 started (TID: 123437093746368)
Thread 2 started (TID: 123437085353664)
Thread 3 started (TID: 123437076960960)

Sending signal to PROCESS
Signal 10 handled by thread 123437096916800

Sending signal to THREAD 3
Signal 10 handled by thread 123437076960960
new-user@new-user-VirtualBox:~$ nano q3.c
new-user@new-user-VirtualBox:~$
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <signal.h>
```

```
#include <unistd.h>
```

```
pthread_t threads[3];
```

```
void handler(int sig) {
```

```
    printf("Signal %d handled by thread %lu\n", sig, pthread_self());
```

```
}
```

```
void* worker(void* arg) {
```

```
    int id = *(int*)arg;
```

```
    signal(SIGUSR1, handler);
```

```
    printf("Thread %d started (TID: %lu)\n", id, pthread_self());
```

```
    while (1) {
```

```
        sleep(1);
```

```
    }
```

```
    return NULL;
}

int main() {
    int ids[3] = {1, 2, 3};

    signal(SIGUSR1, handler);

    for (int i = 0; i < 3; i++) {
        pthread_create(&threads[i], NULL, worker, &ids[i]);
    }

    sleep(2);

    printf("\nSending signal to PROCESS\n");
    kill(getpid(), SIGUSR1);

    sleep(2);

    printf("\nSending signal to THREAD 3\n");
    pthread_kill(threads[2], SIGUSR1);

    sleep(2);

    return 0;
}
```

EXPLANATION:

This program demonstrates signal handling in multithreaded programs. Signals sent using `kill()` are delivered to any thread, while `pthread_kill()` delivers the signal to a specific thread.